

THE VERIFICATION BAR

WEEK 1, DAY 1

Reproducible environments and tooling. Foundation layer.



A model hands you forty lines of Python.
They look completely reasonable.

HOW DO YOU KNOW THEY ARE CORRECT?

09:00

THE SPINE OF THE DAY

REPRODUCIBLE ENVIRONMENTS AND TOOLING

THE ARGUMENT

- 1 machine** An unreproducible machine is a claim nobody can check. Pin it, rebuild it, prove it.
- 2 code** Unverified code is a guess. Lint it, then type it, then test it.
- 3 the bar** That gate is the bar every line of AI-generated code has to clear.

Nothing today involves AI. We will get there :)

WHAT REPRODUCIBLE ACTUALLY MEANS

OBSTACLES AND FIXES

GIVEN ONLY

- Another person.
- Another machine.
- Only what is committed to the repository.

YOU GET

Same behaviour.

Not similar.

The same.

Three obstacles stand in the way, and each has a different fix.

OBSTACLE ONE: ISOLATION

OBSTACLE ONE OF THREE

One system Python. Every project sharing one set of libraries.

THE COLLISION

Project A needs pydantic 1. Project B needs pydantic 2.
You cannot have both.

You have quietly modified the Python your operating system depends on.

HOW TO DETECT IT

```
sys.prefix      -> .venv/  
                (the project)  
sys.base_prefix -> /usr/bin/python3  
                (the machine)
```

```
if sys.prefix == sys.base_prefix:  
    # you are NOT isolated
```

A virtual environment is not tidiness. It is the boundary that makes a project a thing.

OBSTACLE TWO: PINNING

OBSTACLE TWO OF THREE

UNPINNED

`pydantic`

means whatever version is newest at the moment you happen to run install.

Time makes the decision.

PINNED

`pydantic==2.9.2`

is a statement of fact that survives time.

You make the decision.

OBSTACLE THREE: ONE LINE IN, FIVE PACKAGES OUT

OBSTACLE THREE OF THREE

1

ASKED FOR

5

INSTALLED

4

NOBODY CHOSE

UV PIP COMPILER OUTPUT

```
annotated-types==0.8.0      # via pydantic
pydantic==2.13.5            # via requirements
pydantic-core==2.46.5       # via pydantic
typing-extensions==4.16.0   # via pydantic
typing-inspection==0.5.1    # via pydantic
```

Verified with `uv pip compile` against a file containing the single line: `pydantic`

THE THREE FIXES, IN ORDER

ISOLATE, PIN, LOCK, PROVE

IN THIS ORDER

- 1 Isolate** A virtual environment per project.
- 2 Pin** Direct requirements with double equals, stated in exactly one file.
- 3 Lock** The full resolved tree, transitive dependencies included, committed.

Then prove it: delete the environment and rebuild from the repository alone.

DEMO-1 BACKUP: ONE FILE, TWO ENVIRONMENTS

THE SAME FILE, TWO ANSWERS

ENVIRONMENT A PYDANTIC 1.10.13

```
AttributeError: type object  
'Entry' has no attribute  
'model_validate'
```

ENVIRONMENT B PYDANTIC 2.9.2

```
parsed OK:  
reference='SYN-001'  
amount=10.1
```

The file did not change. Whatever happened is not a property of the file.

READING CODE IS NOT VERIFICATION

WHY THE INSTRUMENT IS WRONG

Not because you are careless. Because reading:

FOUR PROPERTIES

- is not repeatable
- does not scale
- gets worse when you are tired
- gets worse when you are rushed

FROM WEEK 2

You review code you did not write, at a volume no human reads carefully.

The claim is not that developers are sloppy. It is that reading is the wrong instrument.

THREE GATES, THREE QUESTIONS

THE GATE A MACHINE RUNS

EACH ASKS SOMETHING DIFFERENT

- | | | |
|----------|---------------|---|
| 1 | ruff | Is this code shaped like working code? |
| 2 | mypy | Do the type claims the author made actually hold? |
| 3 | pytest | Does it do what it says it does? |

Run them in that order. Each one needs the one before it to have passed.

WHAT EACH GATE CANNOT TELL YOU

THREE HOLES, THREE GATES

BLIND SPOTS

1 **ruff**

Blind to: whether it computes the right answer

2 **mypy**

Blind to: whether the claims were the right claims

3 **pytest**

Blind to: anything you did not think to test

Every one has a hole. That is why you run all three.

"Anything you did not think to test" is the whole game with a model. (we will loop back to this in Week 4)

GATE ONE: B006 IS NOT A STYLE RULE

THE MUTABLE DEFAULT

THE SIGNATURE

```
def add_entry(reference, amount, entries=[]):
```

TWO CALLS, ONE LIST

```
call 1: [{'reference': 'SYN-001', 'amount': 10.1}]
```

```
call 2: [{'reference': 'SYN-001', 'amount': 10.1},  
        {'reference': 'SYN-002', 'amount': 20.2}]
```

THE QUESTION

I never passed it the first entry.

Where did it come from?

A default is evaluated once, when Python reads the def line. One list. Shared by every call.

Silent data corruption across calls. If it were adding transactions, it would invent money.

GATE TWO: A TYPE IS AN ENFORCEABLE CLAIM

THE SIGNATURE THAT LIES

THE CODE

```
def find_reference(entries, reference) -> str:
    for entry in entries:
        if entry.reference == reference:
            return reference
```

MYPY

```
error: Missing
return statement
[return]
```

-> **str** promises a string on every path. This one returns None on a miss.

The function lies. The caller will believe it, write `result.upper()`, and fail in production far from this file.

DEFAULT VERSUS STRICT

TWO DIFFERENT KINDS OF FINDING

1

ERROR, DEFAULT MODE

MEANS

you said something false.

16

ERRORS, STRICT MODE

MEANS

you have said almost nothing at all, and I cannot help you until you do.

An unannotated function is invisible to a type checker.



GATE THREE: DO THE SUM

IN YOUR HEAD, OUT LOUD

10.10

20.20

5.05

= ?

THE MACHINE DISAGREES

BINARY FLOATING POINT

PYTEST

```
assert total(book) == 35.35
```

```
AssertionError: assert 35.349999999999994 == 35.35
```

BOTH GATES PASSED

ruff was happy.
mypy was happy.

A correct implementation of the wrong idea.

Fix: `Decimal("10.10")` from a string. Never `Decimal(10.10)`

PRACTICAL: AN UNSEEN MODULE

THIRTY-FIVE MINUTES, ALONE

Thirty-five minutes. Alone. No AI assistance.

THE BRIEF

- 1 build** Build the environment. `environment_doctor.py` must exit 0 before anything else.
- 2 run** Run the three gates in order. Fix only what each one reports.
- 3 test** Add the two missing tests at the bottom of the test file.

Do not read the module hunting for bugs. That is the thing today established does not work.

THE PASS BAR

ALL FIVE, NO PARTIAL CREDIT

1 Environment rebuilds from committed files; the doctor exits 0

2 ruff check . reports: All checks passed!

3 mypy src tests reports: Success: no issues found

4 pytest fully green, including the two tests you wrote

5 Decimal for every monetary value, constructed from strings

All five. No partial credit.

THE BAR DOES NOT MOVE FOR AI

A tool made a confident, correct-looking, locally-valid suggestion.
It was wrong about what you were trying to do.
Most of you accepted it without reading the diff.

Now replace ruff with the model, and one import with forty lines of business logic.

TOMORROW: GIT AND CODE REVIEW.